APPLICATION BRIEF / 07

# Service Booking App

Build a reliable appointment-booking experience for a small service provider and their customers.

| BUILD WINDOW | CORE LOOP | TEAM SIZE |
| --- | --- | --- |
| 4 hours<br>Competition Day | Browse → Book → Manage<br>One shared schedule | Solo or up to 4<br>Collaborative build |

## YOUR MISSION

Create a responsive web application where customers can find available appointment slots, book a time, and review their upcoming bookings. A service provider must be able to view and manage the complete schedule. The app should behave like a connected product, not just a set of static screens.

### 01 / Understand

Turn the brief into a clear plan before you code.

### 02 / Build

Prioritize the required flow and make it work end to end.

### 03 / Ship

Use version control and deploy a working production URL.

### 04 / Explain

Be ready to show how your system works and why.

## Design direction

Use the WebForge visual language as inspiration: dark, technical, editorial, and structured. Think terminal-inspired labels, crisp typography, restrained accent colors, clear hierarchy, and purposeful motion. Do not make the entire interface pink or rely on decoration at the expense of usability. The final visual identity is yours to define.

Competition date: Saturday, 31 October 2026 • 10:00 AM–2:00 PM • In person

# 01 / Product requirements

## Target users

- Customers: people who need to book a service at a suitable time.
- Provider: a small service business such as a tutor, salon, or repair shop that manages availability and appointments.

## Core user journeys

| CUSTOMER | PROVIDER |
| --- | --- |
| 1. Browse available appointment slots.<br>2. Sign in or create an account.<br>3. Book an available slot.<br>4. See upcoming bookings in a personal view.<br>5. If implemented, cancel or reschedule a booking. | 1. Sign in to the provider view.<br>2. See the schedule across customers.<br>3. Review booking details and status.<br>4. Manage bookings clearly and consistently. |

## Required features

- Authentication: booking actions must be tied to the signed-in customer.
- Availability: customers can identify open slots; once a slot is booked, it must no longer appear as available to other customers.
- Personal bookings: each customer can view their own upcoming bookings.
- Provider schedule: a provider-facing view shows bookings across customers.
- Persistent data: bookings and relevant account data remain available after refresh and a new session.
- Responsive UI: core journeys remain usable on mobile and desktop.

## Optional stretch goals

- Allow customers to cancel or reschedule their own bookings.
- Add booking status labels, useful empty states, filters, or clear confirmation feedback.
- Improve accessibility, validation, and loading/error states.

Stretch goals are optional. Do not sacrifice the required booking flow to add extra features.

# 02 / Technical expectations

Use the workshop toolchain and be prepared to explain the choices your team makes.

## Required workflow

| | | |
|---|---|---|
| 01 | **Understand & plan** | Interpret this brief and map the system in Excalidraw. |
| 02 | **Design** | Use Variant to establish the UI/UX direction and key screens. |
| 03 | **Build** | Develop in Visual Studio Code with Codex and/or Claude Code. |
| 04 | **Connect** | Use Firebase for authentication and persistent application data. |
| 05 | **Version** | Track work in GitHub with meaningful commits. |
| 06 | **Deploy** | Publish the application through Vercel. |
| 07 | **Configure** | Connect the domain through Cloudflare. |
| 08 | **Discover & measure** | Set up Google Search Console and Bing Webmaster Tools; evaluate the deployed site with PageSpeed Insights. |

## Planning checklist

- Identify the main data entities and their relationships.
- Decide how you will prevent two customers from booking the same slot.
- Define what a customer is allowed to see or change, and what belongs in the provider view.
- Sketch the main screens and states: loading, empty, success, and error.
- Agree on a minimum working version before dividing stretch work.

Data integrity and access control matter. Do not rely on hiding a button in the UI as your only permission check. Verify that users cannot access or alter another customer's bookings.

## Final deliverables

- A deployed, working application URL.
- A GitHub repository with the project source and meaningful commit history.
- An Excalidraw system diagram that your team can explain.
- A clear final demonstration of the required customer and provider journeys.
- A short explanation of your implementation, key trade-offs, and any known limitations.

# 03 / Adaptability checkpoint

A new client requirement will be announced during the competition. Your team will need to assess its impact, adapt the existing application, and explain your approach.

**CHANGE REQUEST // REVEALED**

# Recurring unavailable time

The provider now needs to block out a recurring unavailable time after bookings already exist. The application must detect and flag conflicts between that new recurring block and bookings already on the schedule.

## What this means for your build

- Provide a way for the provider to define a recurring unavailable period.
- Compare the new unavailable period against existing bookings.
- Clearly flag conflicting bookings so the provider can identify them.
- Keep existing booking data intact; do not silently delete or move appointments.
- Make the conflict state understandable in the interface and explain how it is represented in your data model.

You may choose the exact interaction and implementation. Judges will look for a coherent response to the changed requirement, not just a visual mock-up.

## How to respond

- Read the change carefully and identify which screens, data, logic, and permissions it affects.
- Agree on a minimal, testable implementation before making broad changes.
- Test the new behavior against existing data, including a conflict and a non-conflict case.
- Be ready to explain what changed, what you prioritized, and what you would improve with more time.

# 04 / Judging criteria

Projects are scored out of 100 points. Judges will evaluate the working product, implementation choices, adaptability, and the team's ability to explain its decisions.

| CATEGORY | WHAT JUDGES LOOK FOR | WEIGHT |
|---|---|---|
| Functional Completeness | Does the application meet the provided requirements and function correctly? | 20% |
| Technical Implementation | How effectively were authentication, persistent data, application logic, GitHub, and deployment implemented? | 20% |
| Problem Solving & Adaptability | How effectively did the team debug problems and respond to the unexpected requirement change? | 20% |
| UI/UX & Design | Is the application intuitive, coherent, responsive, and easy to use? | 15% |
| System Understanding & Planning | Can the team explain its Excalidraw diagram and how the major parts of the system interact? | 10% |
| Production Readiness & Performance | Has the application been deployed correctly, connected to a domain, prepared for search discovery, and evaluated with PageSpeed Insights? | 10% |
| Final Demonstration | Can the team clearly demonstrate and explain its solution? | 5% |
| **TOTAL** | | **100 POINTS** |

## Make the demo easy to judge

- Show the main flow live rather than only presenting slides or screenshots.
- Demonstrate that a booked slot is no longer available.
- Show the customer view and the provider schedule.
- Explain your system diagram and key data/access decisions.
- After the change request, show the adapted behavior and at least one test case.

# 05 / Competition-day playbook

The goal is a small, coherent, deployed application. Prioritize a complete working path over a long list of unfinished features.

| | | | |
|---|---|---|---|
| **01 / Understand it** | Read and interpret the requirements. | **02 / Plan it** | Use Excalidraw to map how the system works. |
| **03 / Design it** | Use Variant to define the UI/UX. | **04 / Build it** | Develop in Visual Studio Code with Codex and Claude Code. |
| **05 / Connect it** | Use Firebase for authentication and persistent data. | **06 / Version it** | Manage the project using GitHub. |
| **07 / Deploy it** | Publish the application through Vercel. | **08 / Give it an address** | Configure the domain with Cloudflare. |
| **09 / Make it discoverable** | Configure Google Search Console and Bing Webmaster Tools. | **10 / Measure it** | Analyze the production application with PageSpeed Insights. |
| **11 / Adapt it** | Respond to the unexpected change request. | **12 / Defend it** | Demonstrate and explain the finished application to instructors. |

## Before you submit

- Open the deployed URL in a fresh browser session and verify the core journey.
- Check the layout at mobile and desktop widths.
- Confirm authentication and data persistence work in the deployed environment.
- Check that GitHub contains the latest working version.
- Prepare a concise demo and make sure every team member understands the system.

**BUILD WITH INTENT. TEST THE FLOW. SHIP SOMETHING REAL.**

ACM WebForge 2026 • Code. Construct. Create.

Bring your own laptop. Work collaboratively. Be ready to explain what you built.